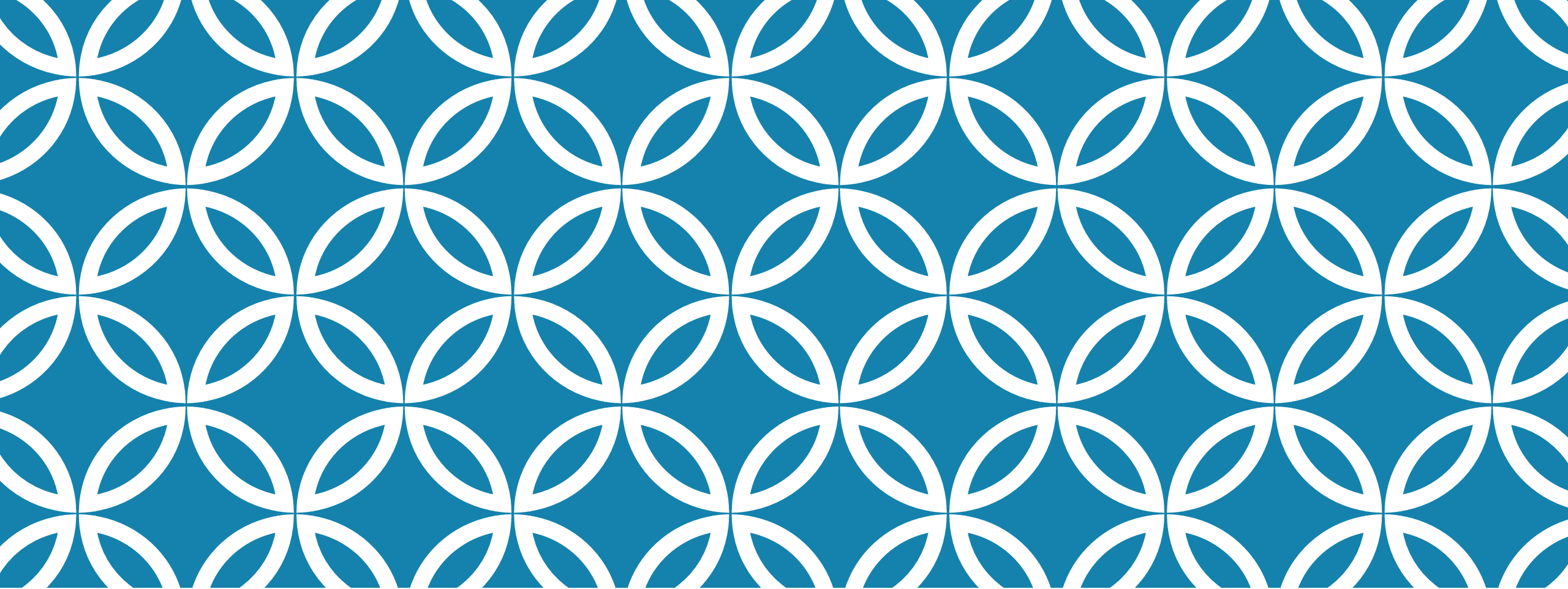




PROGRAMMING IN PYTHON

Gavrilut Dragos
Course 1
(rev.3)

ADMINISTRATIVE

Final grade for the Python course is computed using Gauss over the total points accumulated.

One can accumulate a maximum of 300 of points:

- A lab project (developed between week 8 and week 14) - up to 100 points. Projects selection will be decided in week 8
- Lab activity - maximum of 8 points / lab (starting with lab2) $\Rightarrow 8 \times 6 = 48$ points
- Lab test (week 8) - up to 52 points
- Maximum 100 points at the final examination (course)

The minimum number of points that one needs to pass this exam:

- 120 points summed up from all tests
- 30 points minimum for each category (course, project and lab activity + lab test)

Course page: <https://gdt050579.github.io/python-course-fii/>

HISTORY

1980 – first design of Python language by Guido van Rossum

1989 – implementation of Python language started

2000 – Python 2.0 (garbage collector, Unicode support, etc)

2008 – Python 3.0

2020 – Python 2 is discontinued

Current Versions:

- ❖ 2.x → 2.7.18 (20.Apr.2020)

- ❖ 3.x → 3.12.7 (01.Oct.2024)

Download python from: <https://www.python.org>

Help available at : <https://docs.python.org/3/>

Python coding style: <https://www.python.org/dev/peps/pep-0008/#id32>

GENERAL INFORMATION

- Companies that are using Python: Google, Reddit, Yahoo, NASA, Red Hat, Nokia, IBM, etc
- TIOBE Index for September 2021 → **Python** is ranked no. 2 (Sep. 2021) → <https://www.tiobe.com/tiobe-index/python/>
- Default for Linux and Mac OSX distribution (both 2.x and 3.x versions)
- Open source
- Support for almost everything: web development, mathematical complex computations, graphical interfaces, etc.
- .Net implementation → IronPython (<http://ironpython.net>) for 2.x version

CHARACTERISTICS

- ❖ Un-named type variable
- ❖ Duck typing → type constraints are not checked during compilation phase
- ❖ Anonymous functions (lambda expressions)
- ❖ Design for readability (white-space indentation)
- ❖ Object-oriented programming support
- ❖ Reflection
- ❖ Metaprogramming → the ability to modify itself and create new types during execution

ZEN OF PYTHON

Beautiful is better than ugly.

Explicit is better than implicit.

Simple is better than complex.

Complex is better than complicated.

Flat is better than nested.

Sparse is better than dense.

Readability counts.

Special cases aren't special enough to break the rules.

Although practicality beats purity.

Errors should never pass silently.

In the face of ambiguity, refuse the temptation to guess.

There should be one-- and preferably only one --obvious way to do it.

Although that way may not be obvious at first unless you're Dutch.

Unless explicitly silenced.

Now is better than never.

Although never is often better than *right* now.

If the implementation is hard to explain, it's a bad idea.

If the implementation is easy to explain, it may be a good idea.

Namespaces are one honking great idea -- let's do more of those!

PYTHON EDITORS

Notepad++	➔ https://notepad-plus-plus.org/downloads/v7.7.1/
Komodo IDE	➔ http://komodoide.com
PyCharm	➔ https://www.jetbrains.com/pycharm/
VSCode	➔ https://marketplace.visualstudio.com/items?itemName=donjayamanne.python
Eclipse	➔ http://www.liclipse.com
PyDev	➔ https://wiki.python.org/moin/PyDev
WingWare	➔ http://wingware.com
PyZO	➔ http://www.pyzo.org
Thonny	➔ http://thonny.cs.ut.ee
.....	

FIRST PYTHON PROGRAM

C/C++

```
void main(void)
{
    printf("Hello world");
}
```



Python 3.x

```
print ("Hello world")
```

VARIABLES

Variables are defined and used as you need them.

Python 3.x

```
x = 10           #x is a number
s = "a string"   #s is a string
b = True         #b is a Boolean value
```

Variables don't have a fixed type – during the execution of a program, a variable can have multiple types.

Python 3.x

```
x = 10
#do some operations with x
x = "a string"
#x is now a string
```

[illegible][illegible]

```
Python 3.x
x = 10
print (x, type (x) )
```

```
Python 3.x
```

```
x = 10  
print (x, type (x) )
```

Output

```
10 <class 'int'>
```

Output

```
10 <class 'int'>
```

NUMERICAL OPERATIONS

Arithmetic operators (+, -, *, /, %) – similar to C like languages

Python 3.x

```
x = 10+20*3          #x will be an integer with value 70
x = 10+20*3.0        #x will be a float with value 70.0
```

Operator ****** is equivalent with the pow function from C like languages

Python 3.x

```
x = 2**8              #x will be an integer with value 256
x = 2**8.1            #x will be a float with value 274.374
```

A number can be casted to a specific type using **int** or **float** method

Python 3.x

```
x = int(10.123)       #x will be an integer with value 10
x = float(10)         #x will be a float with value 10.0
```

NUMERICAL OPERATIONS

Division operator has a different behavior in Python 2.x and Python 3.x

Python 3.x

```
x = 10.0/3          #x will be a float with value 3.3333
x = 10.0%3          #x will be a float with value 1.0
```

Division between integers is interpreted differently

Python 2.x

```
x = 10/3
#x is 3 (int)
```

Python 3.x

```
x = 10/3
#x is 3.33333 (float)
```

A special operator exists `//` that means integer division (for integer operators)

Python 3.x

```
x = 10.0//3          #x will be a float with value 3.0
x = 11.9//3          #x will be a float with value 3.0
```

NUMERICAL OPERATIONS

Bit-wise operators (& , | , ^ , << , >>). In particular & operator can be used to make sure that a behavior specific to a C/C++ operation can be achieve

C/C++

```
void main(void)
{
    unsigned int x;
    x = 0xFFFFFFFF;
    x = x + x;
    unsigned char y;
    y = 123;
    y = y + y;
}
```

Python 3.x

```
x = 0xFFFFFFFF
x = (x + x) & 0xFFFFFFFF
y = 123
y = (y + y) & 0xFF
```

NUMERICAL OPERATIONS

Compare operators ($>$, $<$, $>=$, $<=$, $==$, $!=$). C/C++ like operators $\&\&$ and $||$ are replaced with **and** and **OR**. Similar “! operator” is replaced with **not** keyword. However, unlike C/C++ languages Python supports a more mathematical like evaluation.

Python 3.x

```
x = 10 < 20 > 15      #x is True  
                      #identical to (10<20) and (20>15)
```

All of these operators produce a bool result. There are two special values (keywords) defined in Python for constant bool values:

- ❖ True
- ❖ False

STRING TYPES

Python 3.x

```
s = "a string\underline{n}with lines"  
s = 'a string\underline{n}with lines'  
s = r"a string\nwithout any line"  
s = r'a string\nwithout any line'
```

Python 3.x

```
s = """multi-line  
string  
"""
```

Python 3.x

```
s = '''multi-line  
string  
'''
```

STRING TYPES

Strings in python have support for different types of formatting – much like in C/C++ language.

Python 3.x

```
s = "Name: %8s Grade: %d"% ("Ion", 10)
```

If only one parameter has to be replaced, the same expression can be written in a simplified form:

Python 3.x

```
s = "Grade: %d"%10
```

Two special keywords **str** and **repr** can be used to convert variables from any type to string.

Python 3.x

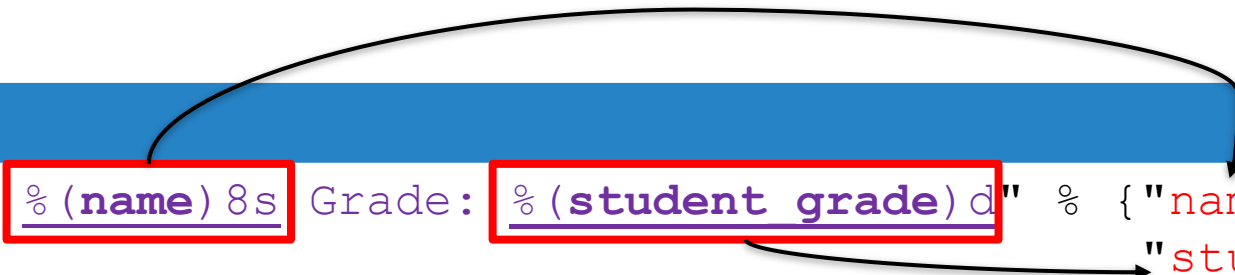
```
s = str (10)           #s is "10"  
s = repr (10.25)       #s is "10.25"
```

STRING TYPES

Formatting can be extended by adding naming to formatting variables.

Python 3.x

```
s = "Name: %(name)8s Grade: %(student grade)d" % {"name": "Ion" ,  
                                                "student_grade": 10}
```



A special character “\” can be placed at the end of the string to concatenate it with another one from the next line.

Python 3.x

```
s = "Python"\  
"Exam"  
#s is "PythonExam"
```

STRING TYPES

Starting with version 3.6, Python also supports formatted string literals. These are strings preceded by an “f” or “F” character

Python 3.6+

```
a = 100
s = f"A = {a}"           #s will be 'A = 100'
s = f"A = {a+10}"        #s will be 'A = 110'
s = f"A = {float(a)}"     #s will be 'A = 100.0'
s = f"A = {float(a):10}"  #s will be 'A =          100.0' (preceded by spaces)
s = f"A = {a#:0x}"        #s will be 'A = 0x64'
```

There are some special characters that can be used to trigger a string representation for an object: !s (means **str**), !r (means **repr**), !a (means **ascii**)

More on this topic: <https://docs.python.org/3/tutorial/inputoutput.html>

STRING TYPES

Strings also support different ways to access characters or substrings

Python 3.x

```
s = "PythonExam"    #s is "PythonExam"

s[1]                #Result is "y" (second character, first index is 0)
s[-1]               #Result is "m" → "PythonExamm" (last character)
s[-2]               #Result is "a" → "PythonExama"
s[:3]               #Result is "Pyt" → "PythonExam" (first 3 characters)
s[4:]               #Result is "onExam" → "PythonExam"
                    # (all the characters starting from the 5th character
                    # of the string until the end of the string)
s[3:5]              #Result is "ho" → "PythonExam" (a substring that
                    # starts from the 3rd character until the 5th one)
s[2:-4]             #Result is "thon" → "PythonExam"
```

STRING TYPES

Strings also support a variety of operators

Python 3.x

```
s = "Python"+"Exam" #s is "PythonExam"
s = "A"+"12"*3      #s is "A121212" → "12" is multiplied 3 times
"A" in "Python"     #Result is False ("A" string does not exists in
                    # "Python" string)
"A" not in "ABC"    #Result is False ("A" string exists in "ABC")
len (s)             #Result is 10 (10 characters in "PythonExam" string)
```

And slicing:

Python 3.x

```
s = "PythonExam"    #s is "PythonExam"
s[1:7:2]             #Result is "yhn" (Going from index 1, to index 7
                    #with step 2 (1,3,5) → PythonExam
```

STRING TYPES

Every string is considered a class and has member functions associated with it. These methods are accessible through “.” operator.

- ❖ **Str.startswith(...)** → checks if a string starts with another one
- ❖ **Str.endswith(...)** → checks if a string ends with another one
- ❖ **Str.replace(toFind,replace,[count])** → returns a string where the substring *<toFind>* is replaced by substring *<replace>*. Count is a optional parameter, if given only the first *<count>* occurrences are replaced
- ❖ **Str.index(toFind)** → returns the index of *<toFind>* in current string
- ❖ **Str.rindex(toFind)** → returns the right most index of *<toFind>* in current string
- ❖ Other functions: **lower()**, **upper()**, **strip()**, **rstrip()**, **lstrip()**, **format()**, **isalpha()**, **isupper()**, **islower()**, **find(...)**, **count(...)**, etc

STRING TYPES

Strings splitting via **.split** function

Python 3.x

```
s = "AB||CD||EF||GH"
s.split("||")[2]    #Result is "EF". Split produces an array of 4
                    #elements AB,CD,EF and GH. The second element is EF
s.split("||")[-1]   #Result is "GH".
s.split("||",1)[0]  #Result is "AB". In this case the second parameter
                    #tells the function to stop after <count> (in this
                    #case 1) splits. Split produces an array of 2
                    #elements AB and CD||EF||GH. The first element is AB
s.split("||",2)[2]  #Result is "EF||GH". Split produces an array of 3
                    #elements AB, CD and EF||GH.
```

Strings also support another function **.rsplit** that is similar to **.split** function with the only difference that the splitting starts from the end and not from the beginning.

BUILT-IN FUNCTIONS FOR STRINGS

Python has several build-in functions design to work characters and strings:

- ❖ **chr** (*charCode*) → returns the string formed from one character corresponding to the code *charCode*. *charCode* is a Unicode code value.
- ❖ **ord** (*character*) → returns the Unicode code corresponding to that specific character
- ❖ **hex** (*number*) → converts a number to a lower-case hex representation
- ❖ **oct** (*number*) → converts a number to a base-8 representation
- ❖ **format** → to format a string with different values

STATEMENTS

Python is heavily based on indentation to express a complex instruction

C/C++

```
if (a>b)
{
    a = a + b
    b = b + a
}
```

Python 3.x

```
if a>b:
    a = a + b
    b = b + a
```

Python 3.x

```
if a>b:
    _____ a = a + b
    _____ b = b + a
```

Complex instruction

STATEMENTS

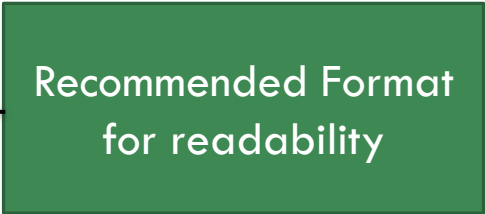
While python coding style recommends using indentation, complex instruction can be written in a different way as well by using a semicolon and add simple expression on the same line:

For example, the following expression:

Python 3.x

```
if a>b:
    a = a + b
    b = b + a
    b = a * b
```

Recommended Format
for readability



Can also be written as follows:

Python 3.x

```
if a>b: a = a + b ; b = b + a ; b = a * b
```

IF-STATEMENT

Python 3.x

```
if expression:  
    complex or simple statement
```

Python 3.x

```
if expression:  
    complex or simple statement  
else:  
    complex or simple statement
```

Python 3.x

```
if expression:  
    complex or simple statement  
elif expression:  
    complex or simple statement
```

Python 3.x

```
if expression:  
    complex or simple statement  
elif expression:  
    complex or simple statement  
elif expression:  
    complex or simple statement  
elif expression:  
    complex or simple statement  
...  
else:  
    complex or simple statement
```

SWITCH/CASE - STATEMENTS

Python (**until 3.10 version**) **does not have** a special keyword to express a switch statement. However, if-elif-else keywords can be used to describe the same behavior.

C/C++

```
switch (var) {  
    case value_1:  
        statements;  
        break;  
    case value_2:  
        statements;  
        break;  
    ...  
    default:  
        statements;  
        break;  
}
```

Python 3.x

```
if var == value_1:  
    complex or simple statement  
elif var == value_2:  
    complex or simple statement  
elif var == value_3:  
    complex or simple statement  
...  
else: #default branch from switch  
    complex or simple statement
```

Python 3.10 match...case statements will be discussed in course no. 2

WHILE - STATEMENT

C/C++

```
while (expression) {  
    statements;  
}
```

Python 3.x

```
while expression:  
    complex or simple statement
```

Python 3.x

```
while expression:  
    complex or simple statement  
else:  
    complex or simple statement
```

Python 3.x

```
a = 3  
while a > 0:  
    a = a - 1  
    print (a)  
else:  
    print ("Done")
```

Output

2
1
0
Done

WHILE - STATEMENT

The **break** keyword can be used to exit the while loop. Using the **break** keyword will not move the execution to the **else** statement if present !

Python 3.x

```
a = 3
while a > 0:
    a = a - 1
    print (a)
    if a==2: break
else:
    print ("Done")
```

Output

2

WHILE - STATEMENT

Similarly, the **continue** keyword can be used to switch the execution from the while loop to the point where the while condition is tested.

Python 3.x

```
a = 10
while a > 0:
    a = a - 1
    if a % 2 == 0: continue
    print (a)
else:
    print ("Done")
```

Output

9
7
5
3
1
Done

DO...WHILE - STATEMENT

Python **does not have** a special keyword to express a do ... while statement. However, using the **while...else** statement a similar behavior can be achieved.

C/C++

```
do {  
    statements;  
}  
while (test_condition);
```

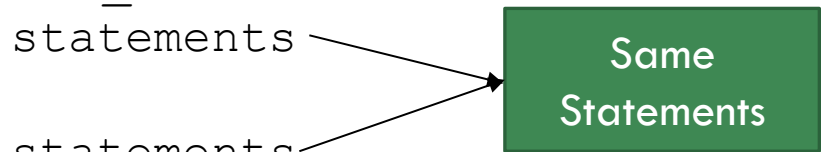
Example:

C/C++

```
do {  
    x = x - 1;  
}  
while (x > 10);
```

Python 3.x

```
while test_condition:  
    statements  
else:  
    statements
```



```
graph LR; A[statements] --> C[Same Statements]; B[statements] --> C;
```

Python 3.x

```
while x > 10:  
    x = x - 1  
else:  
    x = x - 1
```

FOR- STATEMENT

For statement is different in Python than the one mostly used in C/C++ like languages. It resembles more a foreach statement (in terms that it only iterates through a list of objects, values, etc). Besides this, all of the other known keywords associated with a for (**break** and **continue**) work in a similar way.

Python 3.x

```
for <list_of_iterators_variables> in <list>:  
    complex or simple statement
```

Python 3.x

```
for <list_of_iterators_variables> in <list>:  
    complex or simple statement  
else:  
    complex or simple statement
```

FOR- STATEMENT

A special keyword **range** that can be used to simulate a C/C++ like behavior.

Python 3.x

```
for index in range (0,3):  
    print (index)
```

Output

0
1
2

Python 3.x

```
for index in range (0,3):  
    print (index)  
else:  
    print ("Done")
```

Output

0
1
2
Done

FOR- STATEMENT

range operator in Python 3.x returns an iterable object

range is declared as follows **range** (*start*, *end*, [*step*])

Python 3.x

```
for index in range (0, 8, 3):  
    print (index)
```

Output

0
3
6

for statement will be further discuss in the course no. 2 after the concept of list is presented.

FUNCTIONS

Functions in Python are defined using **def** keyword

Python 3.x

```
def function_name (param1,param2,... paramn ) :  
    complex or simple statement
```

Parameters can have a default value.

Python 3.x

```
def function_name (param1,param2 [= defaultVal],... paramn[= defaultVal] ) :  
    complex or simple statement
```

And finally, **return** keyword can be used to return values from a function. There is no notion of void function (similar cu C/C++ language) → however, this behavior can be duplicated by NOT using the **return** keyword.

FUNCTIONS

Example of a function that performs a simple arithmetic operation

Python 3.x

```
def myFunc (x, y, z):  
    return x * 100 + y * 10 + z  
print ( myFunc (1,2,3) )
```

#Output:123

Parameters can be explicitly called

Python 3.x

```
def sum (x, y, z):  
    return x * 100 + y * 10 + z  
print ( sum (z=1, y=2, x=3) )
```

#Output:321

FUNCTIONS

Function parameters can have default values. Once a parameter is defined using a default value, every parameter that is declared after it should have default values.

Python 3.x

```
def myFunc (x, y=6, z=7) :  
    return x * 100 + y * 10 + z  
print (myFunc (1) )  
print (myFunc (2, 9) )  
print (myFunc (z=5, x=3) )  
print (myFunc (4, z=3) )  
print (myFunc (z=5) )
```

#Output:167
#Output:297
#Output:365
#Output:463
#ERROR: **missing x**

Python 3.x

```
def myFunc (x=2, y, z=7) :  
    return x * 100 + y * 10 + z
```

**Code will not compile as x
has a default value, but Y
does not !**

FUNCTIONS

A function can return multiple values at once. This will also be discussed in course no. 2 along with the concept of tuple.

Python also uses **global** keyword to specify within a function that a specific variable is in fact a global variable.

Python 3.x

```
x = 10
def ModifyX ():
    x = 100
ModifyX ()
print ( x ) #Output:10
```

Python 3.x

```
x = 10
def ModifyX ():
    global x
    x = 100
ModifyX ()
print ( x ) #Output:100
```

FUNCTIONS

Functions can have a variable – length parameter (similar to the ... from C/C++). It is preceded by “*” operator.

Python 3.x

```
def multi_sum (*list_of_numbers):  
    s = 0  
    for number in list_of_numbers:  
        s += number  
    return s  
  
print ( multi_sum (1,2,3) )           #Output:6  
print ( multi_sum (1,2) )             #Output:3  
print ( multi_sum (1) )               #Output:1  
print ( multi_sum () )                #Output:0
```

FUNCTIONS

Functions can return values of different types. In this case you should check the type before using the return value.

Python 3.x

```
def myFunction(x) :  
    if x>0:  
        return "Positive"  
    elif x<0:  
        return "Negative"  
    else:  
        return 0  
result = myFunction (0)  
if type(result) is int:  
    print("Zero")  
else:  
    print(result)
```

FUNCTIONS

Functions can also contain another function embedded into their body.

That function can be used to compute results needed in the first function.

Python 3.x

```
def myFunction(x) :  
    def add (x, y) :  
        return x+y  
    def sub (x, y) :  
        return x-y  
  
    return add(x, x+1) + sub(x, 2)  
print (myFunction (5))
```

The previous code will print 14 into the screen.

FUNCTIONS

Functions can also be recursive (see the following implementation for computing a Fibonacci number)

Python 3.x

```
def Fibonacci (n) :  
    if n == 1:  
        return 1  
    elif n == 2:  
        return 1  
    else:  
        return Fibonacci (n-1) + Fibonacci (n-2)  
  
print ( Fibonacci (10) )
```

The previous code will print 55 into the screen.

FUNCTIONS

It is recommended to add a short explanation for every defined function by adding a multi-line string immediately after the function definition

<https://www.python.org/dev/peps/pep-0257/#id15>

Python 3.x

```
def Fibonacci (n) :  
    """  
    Computes the n-th Fibonacci number using recursive calls  
    """  
    if n == 1:  
        return 1  
    elif n == 2:  
        return 1  
    else:  
        return Fibonacci (n-1) + Fibonacci (n-2)
```

HOW TO CREATE A PYTHON FILE

- ❖ Create a file with the extension .py
- ❖ If you run on a Linux/OSX operation system, you can add the following line at the beginning of the file (the first line of the file):
 - ❖ `#!/usr/bin/python3` → for python 3
 - ❖ `#!/usr/bin/python` → for python (current version – usually 2)
- ❖ These lines can be added for windows as well (“#” character means comment in python so they don’t affect the execution of the file too much
- ❖ Write the python code into the file
- ❖ Execute the file.
 - ❖ You can use the python interpreter directly (usually C:\Python27\python.exe or C:\Python310\python.exe for Windows) and pass the file as a parameter
 - ❖ Current distributions of python make some associations between .py files and their interpreter. In this cases you should be able to run the file directly without using the python executable.